axiateam
RDT company

Anexo 5A

SSR: Frameworks para Rendering en Backend

Curso: TypeScript

90 min

Objetivos de Aprendizaje

- Comprender que Webpack NO hace SSR - solo empaqueta
- Conocer los frameworks modernos para SSR: Next.js, Remix, Astro
- Evaluar cuando migrar de Webpack SPA a SSR
- Aplicar el patrón correcto según el tipo de aplicación
- Migrar de SPA Webpack a Next.js SSR
- Tipar Server Components con params y searchParams
- Implementar Data Fetching con tipos y Zod
- Usar Streaming y Suspense con tipos
- Configurar Metadata API para SEO
- Crear API Routes avanzados con validación

1. Contexto: El problema del cliente

10 minutos

Escenario: "Se renderiza en el back"

La confusión común:

- "Uso Webpack" → pero Webpack solo empaqueta, NO hace SSR
- "Quiero renderizar en backend" → necesitas un FRAMEWORK

Lo que el cliente ACTUALMENTE tiene:

Webpack → empaqueta JS → navegador → React hydrate → render (NO es SSR)

Webpack != SSR

Herramienta	Tipo	Hace SSR?
Webpack	Bundler	NO
Vite	Dev server + Bundler	NO
Next.js	Framework	SI
Remix	Framework	SI
Astro	Framework	SI

2. Webpack != SSR

10 minutos

Cuándo Migrar a Next.js

Señal	Acción
Contenido cambia frecuentemente	Migrar a SSR
SEO es importante	Migrar a SSR
Primer paint lenta	Migrar a SSR
App muy interactiva (dashboards)	Quedarse en SPA

Beneficios: SEO mejorado, FCP más rápido, mejor UX

3. Next.js

15 minutos

¿Por qué Next.js?

- Más popular (20M+ descargas/semana)
- Usa Webpack internamente (o Turbopack)
- Vercel detrás
- App Router (React Server Components)
- API Routes integradas

Crear proyecto Next.js

```
npx create-next-app@latest my-app --typescript
```

Lab: Crear proyecto Next.js básico

Objetivo: Crear una app Next.js con SSR desde cero

1. Ejecutar npx create-next-app@latest
2. Configurar: App Router, TypeScript
3. Ejecutar npm run dev
4. Verificar View Source en navegador

4. Server vs Client Components

15 minutos

La regla de oro

En Next.js App Router, los componentes son Server Components por defecto.

Característica	Server	Client
Renderizado	Servidor	Cliente
Hooks (useState, useEffect)	No	Sí
Event handlers	No	Sí
Acceso BD/FS	Sí	No

Árbol de decisiones

```
¿Tiene interactividad? (useState, onClick)
  → SI → Client Component ('use client')
  → NO → ¿Accede a BD/FS?
      → SI → Server Component
      → NO → Server Component (por defecto)
```

Lab: Migrar componente a Server Component

Objetivo: Convertir un componente de SPA en Server Component

1. Analizar qué necesita el componente
2. Si no tiene interactividad → Server Component
3. Si tiene useState/useEffect → 'use client'

5. Routing y API Routes

15 minutos

Routing: app/tasks/[id]/page.tsx

```
interface PageParams {
  params: { id: string };
}

export default async function TaskPage({ params }: PageParams) {
  const task = await getTask(params.id);
  return <TaskDetail task={task} />;
}
```

Route Handlers: app/api/tasks/route.ts

```
export async function GET() {
  const tasks = await getTasks();
  return NextResponse.json(tasks);
}

export async function POST(request: NextRequest) {
  const body = await request.json();
  const created = await createTask(body);
  return NextResponse.json(created, { status: 201 });
}
```

Lab: Crear rutas y API en Next.js

Objetivo: Crear routing y API para la app de tareas

1. Crear app/tasks/page.tsx
2. Crear app/tasks/[id]/page.tsx
3. Crear app/api/tasks/route.ts

6. Tipado de Server Components

15 minutos

Params como Promise (Next.js 15+)

```
interface PageProps {
  params: Promise<{ id: string }>;
}

export default async function TaskPage({ params }: PageProps) {
  const { id } = await params;
  const task = await getTask(id);
  return <TaskDetail task={task} />;
}
```

SearchParams tipado

```
interface PageSearchParams {
  searchParams: Promise<{ page?: string; filter?: string }>;
}

export default async function TasksPage({ searchParams }: PageSearchParams) {
  const { page, filter } = await searchParams;
  // ...
}
```

Lab: Tipar Server Components

Objetivo: Crear Server Components con tipos correctos

1. Crear página con params tipados
2. Añadir searchParams
3. Verificar sin errores de TypeScript

7. Data Fetching con Tipos

15 minutos

Fetch tipado + Zod

```
import { z } from 'zod';

const TaskSchema = z.object({
  id: z.number(),
  title: z.string(),
  status: z.enum(['pending', 'completed']),
});

type Task = z.infer<typeof TaskSchema>;

async function getTasks(): Promise<Task[]> {
  const res = await fetch('/api/tasks');
  return res.json();
}
```

Beneficio: Un solo schema = tipos en frontend y backend

Lab: Data Fetching Tipado con Zod

Objetivo: Implementar fetch y API routes con tipos y validación Zod

1. Instalar Zod
2. Crear tipos en types/task.ts
3. Crear API route con validación

8. Streaming y Suspense

15 minutos

Archivos especiales

Archivo	Cuando se muestra
loading.tsx	Mientras carga la página
error.tsx	Cuando hay error
not-found.tsx	Cuando notFound() es llamado

Suspense

```
import { Suspense } from 'react';

export default function Page() {
  return (
    <Suspense fallback={<Skeleton />}>
      <TaskList />
    </Suspense>
  );
}
```

Lab: Implementar Streaming y Suspense

Objetivo: Añadir loading, error, not-found y Suspense

1. Crear app/tasks/loading.tsx
2. Crear app/tasks/error.tsx
3. Crear app/tasks/not-found.tsx

9. Metadata API

10 minutos

Metadata estática

```
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Task Manager',
  description: 'Lista completa de tareas',
  openGraph: {
    title: 'Task Manager',
    type: 'website',
  },
};
```

Metadata dinámica

```
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const task = await getTask(id);

  return {
    title: task ? `${task.title} - Task Manager` : 'Not found',
  };
}
```

Lab: Metadata API con Tipos

Objetivo: Implementar metadata estática y dinámica

1. Añadir metadata en layout.tsx
2. Añadir generateMetadata en páginas
3. Verificar en DevTools

10. API Routes Avanzados

15 minutos

Headers, Cookies y Query Params

```
export async function GET(request: NextRequest) {
  const authHeader = request.headers.get('authorization');
  const sessionCookie = request.cookies.get('session');
  const { searchParams } = new URL(request.url);
  const page = searchParams.get('page');

  return NextResponse.json({ auth, session, page });
}
```

Lab: API Routes Completos

Objetivo: Crear API routes con headers, cookies, query params y validación

1. Crear app/api/tasks/route.ts
2. Crear app/api/tasks/[id]/route.ts
3. Probar endpoints

11. Ejercicio Integrador

Migrar Caso 2r (Biblioteca) a Next.js SSR

Requisitos

- Crear proyecto Next.js con la app
- Convertir componentes existentes a Server/Client Components
- Migrar routing a estructura Next.js App Router
- Crear páginas necesarias
- Implementar API routes para CRUD
- Configurar variables de entorno

12. Resumen

5 minutos

Puntos Clave

1. Webpack NO hace SSR - solo empaqueta
2. SSR = Framework: Next.js, Remix, Astro
3. Server vs Client Components - Server por defecto, use client para interactividad
4. Routing - estructura de carpetas en app/
5. API Routes - crear endpoints sin backend separado
6. Tipado de Server Components - params como Promise, searchParams
7. Data Fetching - fetch tipado, Zod para validación
8. Streaming - Suspense, loading.tsx, error.tsx, not-found.tsx
9. Metadata API - SEO con tipos estáticos y dinámicos
10. API Routes avanzados - headers, cookies, query params con tipos

13. Referencias

Recursos Adicionales

- Next.js Documentation
- Next.js App Router
- React Server Components
- SSR vs SSG
- Next.js Metadata API
- Zod Validation

Gracias!

Anexo 5A Completado

Siguiente: Anexo 8A - Patrones TS Avanzados